

Основы языка программирования Python¹

Модули

```
# application.py
```

```
import tools
```

```
foo = tools.add(10, 20)
```

```
bar = tools.mul(4, 2)
```

```
print(f"{tools=}")
```

```
print(f"{type(tools)=}")
```

```
print(f"{dir(tools)=}")
```

```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

tools=<module 'tools' from '../tools.py'>

type(tools)=<class 'module'>

dir(tools)=['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'add', 'mul']

```
# application.py
```

```
import tools as tl
```

```
foo = tl.add(10, 20)
```

```
bar = tl.mul(4, 2)
```

```
print(f"{tl=}")
```

```
print(f"{type(tl)=}")
```

```
print(f"{dir(tl)=}")
```

```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

```
tl=<module 'tools' from '.../tools.py'>
```

```
type(tl)=<class 'module'>
```

```
dir(tl)=['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__',  
        '__package__', '__spec__', 'add', 'mul']
```

```
# application.py
```

```
from tools import add, mul
```

```
foo = add(10, 20)
```

```
bar = mul(4, 2)
```

```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

```
# application.py
```

```
from math import sqrt
from cmath import sqrt as csqrt
```

```
from tools import add, mul
```

```
foo = sqrt(add(10, 20))
bar = csqrt(mul(4, -2))
```

```
# tools.py
```

```
def add(a, b):
    return a + b
```

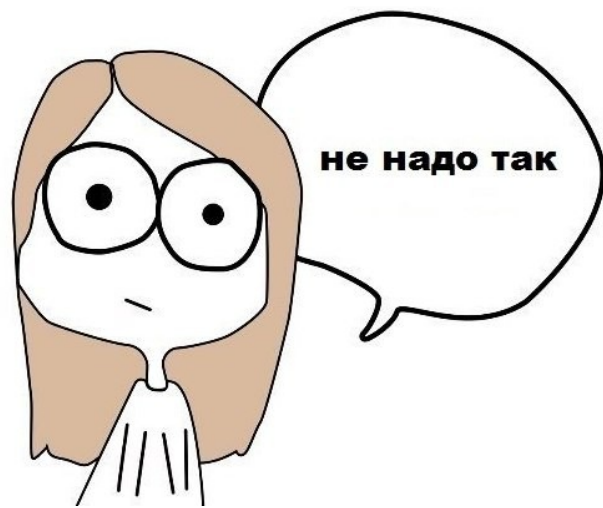
```
def mul(a, b):
    return a * b
```

```
# application.py
```

```
from tools import *
```

```
foo = add(10, 20)
```

```
bar = mul(4, 2)
```



```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

```
# application.py
```

```
print("Внутри application.py - 1")
```

```
import tools
```

```
print("Внутри application.py - 2")
```

```
foo = tools.add(10, 20)
```

```
bar = tools.mul(4, 2)
```

```
print("Внутри application.py - 3")
```

```
# tools.py
```

```
print("Внутри tools.py - 1")
```

```
def add(a, b):  
    return a + b
```

```
print("Внутри tools.py - 2")
```

```
def mul(a, b):  
    return a * b
```

```
> python application.py
```

```
Внутри application.py - 1
```

```
Внутри tools.py - 1
```

```
Внутри tools.py - 2
```

```
Внутри application.py - 2
```

```
Внутри application.py - 3
```

```
# application.py
```

```
import tools
```

```
foo = tools.add(10, 20)
```

```
bar = tools.mul(4, 2)
```

```
print("Внутри application.py:", f"__name__={}")
```

```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

```
print("Внутри tools.py:", f"__name__={}")
```

```
> python application.py
```

```
Внутри tools.py: __name__='tools'
```

```
Внутри application.py: __name__='__main__'
```

```
# application.py
```

```
import tools
```

```
if __name__ == "__main__":  
    foo = tools.add(10, 20)  
    bar = tools.mul(4, 2)  
    print("Внутри application.py:", f"__name__={__name__}")
```

```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

```
if __name__ == "__main__":  
    print("Внутри tools.py:", f"__name__={__name__}")
```

```
> python application.py
```

```
Внутри application.py: __name__='__main__'
```

```
# application.py
```

```
import tools
```

```
if __name__ == "__main__":  
    foo = tools.add(10, 20)  
    bar = tools.mul(4, 2)  
    print("Внутри application.py:", f"__name__={__name__}")  
    print("Внутри application.py:", f"__file__={__file__}")  
    print("Внутри application.py:", f"tools.__file__={tools.__file__}")
```

```
# tools.py
```

```
def add(a, b):  
    return a + b
```

```
def mul(a, b):  
    return a * b
```

```
print("Внутри tools.py:", f"__name__={__name__}")  
print("Внутри tools.py:", f"__file__={__file__}")
```

```
> python application.py
```

```
Внутри tools.py:      __name__='tools'  
Внутри tools.py:      __file__='.../tools.py'  
Внутри application.py: __name__='__main__'  
Внутри application.py: __file__='.../application.py'  
Внутри application.py: tools.__file__='.../tools.py'
```

Пакеты модулей

```
src/12. Modules/example_modules_10.py  
src/12. Modules/example_modules_11.py  
src/12. Modules/example_modules_12.py
```

Установка библиотек с помощью pip

https://pypi.org



[Помощь](#) [Спонсоры](#) [Войти](#) [Зарегистрироваться](#)

Фильтр по классификатору 10 000+ проектов по запросу «numpy» Сортировать по

Соответствию ▾

▼ Framework

▼ Topic

▼ Development Status

▼ License

▼ Programming Language

▼ Operating System

▼ Environment

▼ Intended Audience

▼ Natural Language

▼ Typing

 **numpy** 16 мар. 2025 г.
Fundamental package for array computing in Python

 **numpy-2425** 1 апр. 2025 г.
A custom NLP package

 **video2numpy** 11 сент. 2023 г.
Video reading into numpy

 **wandb2numpy** 7 дек. 2022 г.
Library to export data from WandB to numpy arrays or csv files.

 **x256numpy** 16 янв. 2025 г.
Quickly find the nearest xterm 256 color index of an RGB (NumPy Version)

 **lapjv-numpy2** 19 июн. 2024 г.
Linear sum assignment problem solver using Jonker-Volgenant algorithm. Fork to support numpy 2.x

Установка библиотек с помощью pip

14

Установка пакетов для всех пользователей ОС
(требуется права администратора):

```
> pip install пакет_1 пакет_2 ... пакет_N
```

ИЛИ

```
> python -m pip install пакет_1 пакет_2 ... пакет_N
```

Установка библиотек с помощью pip

15

Установка пакетов для текущего пользователя ОС
(права администратора не требуются):

```
> pip install --user пакет_1 пакет_2 ... пакет_N
```

ИЛИ

```
> python -m pip install --user пакет_1 пакет_2 ... пакет_N
```

Получение списка установленных библиотек с помощью pip

16

Получить список всех установленных пакетов:

```
> pip list
```

Получить список установленных пакетов, для которых
появились новые версии:

```
> pip list --outdated
```

Обновление библиотек с помощью pip

Обновление библиотек для всех пользователей ОС
(требуется права администратора):

```
> pip install --upgrade пакет_1 пакет_2 ... пакет_N
```

Обновление библиотек для текущего пользователя ОС
(права администратора не требуются):

```
> pip install --upgrade --user пакет_1 пакет_2 ... пакет_N
```

Удаление библиотек с помощью `pip`

```
> pip uninstall пакет_1 пакет_2 ... пакет_N
```

Установка библиотек для инженерных и научных вычислений

19

```
> pip install --user numpy matplotlib scipy pandas
```

Файл зависимостей requirements.txt

Пример содержимого файла requirements.txt:

```
numpy  
pandas==2.2.3  
matplotlib==3.10.3
```

Установка пакетов из файла requirements.txt:

```
> python -m pip install --user -r requirements.txt
```

Способы задания номеров версий пакетов

<https://packaging.python.org/en/latest/specifications/version-specifiers/#id5>

Строгое задание номера версии:

```
matplotlib==3.10.3
```

Всегда будет устанавливаться указанная версия или возникать ошибка, если эту версию установить невозможно.

Задание номера версии с использованием маски:

```
matplotlib==3.10.*
```

Будет установлена наиболее новая версия, соответствующая маске.

В данном случае могут быть установлены версии 3.10.0, 3.10.1 и т. д., но не 3.9.x или 3.11.x и т.д.

Задание минимального номера версии, включая указанную:

```
matplotlib>=3.10.3
```

Будет установлена наиболее новая версия не меньше указанной.

В данном случае могут быть установлены версии 3.10.3, 3.10.4, 3.11.x, 4.x.x и т.д.

Задание минимального номера версии, **не** включая указанную:

```
matplotlib>3.10.3
```

Будет установлена наиболее новая версия не меньше указанной.

В данном случае могут быть установлены версии 3.10.4, 3.10.5, 3.11.x, 4.x.x и т.д.

Задание максимального номера версии, включая указанную:

```
matplotlib<=3.10.3
```

Будет установлена наиболее новая версия не больше указанной.

В данном случае могут быть установлены версии 3.10.3, 3.10.2, 3.9.x, 2.x.x и т.д.

Задание максимального номера версии, **не** включая указанную:

```
matplotlib<3.10.3
```

Будет установлена наиболее новая версия не больше указанной.

В данном случае могут быть установлены версии 3.10.2, 3.10.1, 3.9.x, 2.x.x и т.д.

Установка совместимой версии:

```
matplotlib~3.10.3
```

Всегда будет устанавливаться наиболее новая версия, отличающаяся от указанной последним номером, но не меньше указанного номера.

В данном случае могут быть установлены версии 3.10.3, 3.10.4 и т. д., но не 3.10.2 или 3.11.0.

Получение списка установленных пакетов

```
> python -m pip freeze
```

Перенаправление вывода в файл requirements.txt:

```
> python -m pip freeze > requirements.txt
```

```
contourpy==1.3.2  
cycller==0.12.1  
fonttools==4.58.0  
kiwisolver==1.4.8  
matplotlib==3.10.3  
numpy==2.2.5  
packaging==25.0  
pandas==2.2.3  
pillow==11.2.1  
pyparsing==3.2.3  
python-dateutil==2.9.0.post0
```

Виртуальные окружения

Определение путей до библиотек

```
>>> import sys  
>>> sys.path
```

Пример выполнения в Windows:

```
C:\Program Files\Python313\python313.zip  
C:\Program Files\Python313\DLLs  
C:\Program Files\Python313\Lib  
C:\Program Files\Python313  
C:\Users\USERNAME\AppData\Roaming\Python\Python313\site-packages  
C:\Users\USERNAME\AppData\Roaming\Python\Python313\site-packages\win32  
C:\Users\USERNAME\AppData\Roaming\Python\Python313\site-packages\win32\lib  
C:\Users\USERNAME\AppData\Roaming\Python\Python313\site-packages\Pythonwin  
C:\Program Files\Python313\Lib\site-packages
```

Получение информации о пакете

```
> python -m pip show matplotlib
```

```
Name: matplotlib
```

```
Version: 3.10.3
```

```
Summary: Python plotting package
```

```
Home-page: https://matplotlib.org
```

```
Author: John D. Hunter, Michael Droettboom
```

```
Author-email: Unknown <matplotlib-users@python.org>
```

```
License: License agreement for matplotlib versions 1.3.0 and later
```

```
=====
```

```
...
```

```
Location: C:\Users\USERNAME\AppData\Roaming\Python\Python313\site-packages
```

```
Requires: contourpy, cycler, fonttools, kiwisolver, numpy, packaging, pillow
```

```
Required-by:
```

Создание виртуального окружения

```
> python -m venv <путь>
```

<путь> — путь до директории, где будет создано виртуальное окружение.

Пример создания виртуального окружения под Windows

34

```
> python -m venv c:\projects\venv\myproject
```

```
myproject
```

```
├ Include
```

```
├ Lib
```

```
|   └ site-packages
```

```
|       └ pip
```

```
|       └ pip-24.3.1.dist-info
```

```
├ pyenv.config
```

```
└ Scripts
```

```
    ├── activate
```

```
    ├── activate.bat
```

```
    ├── activate.fish
```

```
    ├── Activate.ps1
```

```
    ├── deactivate.bat
```

```
    ├── pip.exe
```

```
    ├── pip3.13.exe
```

```
    ├── pip3.exe
```

```
    ├── python.exe
```

```
    └ pythonw.exe
```

Активация виртуального окружения под Windows

35

```
> <путь>\Scripts\activate.bat
```

Например:

```
> c:\projects\venv\myproject\Scripts\activate.bat
```

Проверим пути для установки пакетов

```
> c:\projects\venv\myproject\Scripts\activate.bat
> python
>>> import sys
>>> for path in sys.path:
...     print(path)
...
C:\Program Files\Python313\python313.zip
C:\Program Files\Python313\DLLs
C:\Program Files\Python313\Lib
C:\Program Files\Python313
c:\projects\venv\myproject
c:\projects\venv\myproject\Lib\site-packages
```

uv

<https://docs.astral.sh/uv/>

Установка uv под Windows

```
PS> powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"
```

Или (не рекомендуемый способ):

```
> python -m pip install --user uv
```

<https://docs.astral.sh/uv/#installation>

> uv

An extremely fast Python package manager.

Usage: `uv [OPTIONS] <COMMAND>`

Commands:

<code>auth</code>	Manage authentication
<code>run</code>	Run a command or script
<code>init</code>	Create a new project
<code>add</code>	Add dependencies to the project
<code>remove</code>	Remove dependencies from the project
<code>version</code>	Read or update the project's version
<code>sync</code>	Update the project's environment
<code>lock</code>	Update the project's lockfile
<code>export</code>	Export the project's lockfile to an alternate format
<code>tree</code>	Display the project's dependency tree
<code>format</code>	Format Python code in the project
<code>tool</code>	Run and install commands provided by Python packages
<code>python</code>	Manage Python versions and installations
<code>pip</code>	Manage Python packages with a pip-compatible interface
<code>venv</code>	Create a virtual environment
<code>build</code>	Build Python packages into source distributions and wheels
<code>publish</code>	Upload distributions to an index
<code>cache</code>	Manage uv's cache
<code>self</code>	Manage the uv executable
<code>help</code>	Display documentation for a command